



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MINOR PROJECT MID-TERM PROGRESS REPORT
ON
MULTI-AGENT DATA ANALYSIS AND INSIGHT GENERATION PLATFORM FOR
FINANCIAL AND SALES DATA**

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, Institute of Engineering, Tribhuvan University, for providing us the opportunity to work on this minor project.

We are deeply indebted to our supervisor, Er. Rajad Shakya, for his guidance, constructive feedback, and continuous encouragement throughout the development of this project. His suggestions have helped us refine both the technical design and the report structure.

We also thank the faculty members of the department for creating an academic environment that supports practical engineering work. Finally, we are grateful to our friends and families for their patience and support during this phase of the project.

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

July 2026

ABSTRACT

This project develops a multi-agent data analysis pipeline that turns raw CSV-based financial and sales data into structured insights and charts. The system is designed around a shared state model and a planned sequence of six specialized agents: structural profiling, semantic tagging, preprocessing, statistical analysis, validation, and report generation. Each agent performs one focused responsibility and passes its results to the next stage through a controlled pipeline.

At the mid-point of development, the first four agents are implemented and the backend pipeline runs end to end from ingestion through statistical analysis. The codebase now includes robust CSV and Excel loading, semantic metadata generation, deterministic preprocessing, and statistical analysis with chart generation. Output validation (Agent 5) and report generation (Agent 6) are not yet implemented and form the main remaining work, together with refinement, testing, and evaluation on more datasets. The project is moving toward a practical analytics tool that can reduce manual effort and make technical analysis more accessible to non-technical users.

Keywords: Multi-agent systems, data analysis, financial reporting, sales analytics, LangGraph, preprocessing, validation, automated reporting

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Definition	1
1.4 Objectives	2
1.5 Scope and Applications	2
2 LITERATURE REVIEW	4
2.1 Traditional Data Analytics Platforms	4
2.2 AI-Assisted Analytics and Large Language Model-Based Data Analysis	4
2.3 Automated Data Visualization from Tabular/CSV Data	5
2.4 Agentic AI Frameworks and Multi-Agent Systems	5
2.5 Existing Research on Automated Data Analysis Systems	6
2.6 Research Gap and Project Positioning	6
3 REQUIREMENT ANALYSIS	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	7
3.3 Requirement Status	8
4 THEORETICAL APPROACH	9
4.1 Large Language Model Theory	9
4.1.1 Decoder-Only Transformer Architecture	9
4.1.2 Tokenization and Embedding	9

4.1.3	Self-Attention	10
4.1.4	Next-Token Prediction	10
4.2	Groq API and LLM Inference Architecture	11
4.2.1	Why the Groq API Is Used	12
4.2.2	The Llama 3.3 70B Model	12
4.2.3	Agent 2 API Call and Prompt Processing	14
5	SYSTEM ARCHITECTURE AND METHODOLOGY	16
5.1	Overall Pipeline	16
5.2	Shared State Design	17
5.3	Methodology	18
6	IMPLEMENTATION DETAILS	19
6.1	Agent 1: Structural Profiler	19
6.1.1	Overview and Purpose	19
6.1.2	Inputs and Outputs	19
6.1.3	Processing Workflow	19
6.1.4	Key Functions	19
6.1.5	Reliability and Pipeline Interaction	20
6.2	Agent 2: Semantic Tagger	21
6.2.1	Overview and Purpose	21
6.2.2	Inputs and Outputs	21
6.2.3	Processing Workflow	21
6.2.4	Key Functions	23
6.2.5	Reliability and Pipeline Interaction	23
6.3	Agent 3: Preprocessor	24
6.3.1	Overview and Purpose	24
6.3.2	Inputs and Outputs	24
6.3.3	Profile Selection	24
6.3.4	Processing Workflow	24
6.3.5	Key Functions	25
6.3.6	Validation and Data Quality	26
6.3.7	Reliability and Pipeline Interaction	27
6.4	Agent 4: Statistical Analysis	27
6.4.1	Overview and Purpose	27
6.4.2	Inputs and Outputs	27
6.4.3	Processing Workflow	27
6.4.4	Key Functions	28
6.4.5	Reliability and Pipeline Interaction	29

6.5	Current Implementation Snapshot	30
7	RESULTS AND ANALYSIS	31
7.1	Observed Progress	31
7.2	Practical Output	31
7.3	Analysis	31
7.4	Current Challenges	32
8	REMAINING TASKS	33
9	CONCLUSION	34
	APPENDICES	35
A	COST ESTIMATE	35
B	PROJECT TIMELINE	36

LIST OF FIGURES

4.1	Decoder-only Transformer architecture used by modern LLMs	11
4.2	Llama 3.3-70B architecture: decoder-only Transformer with GQA, RoPE, SwiGLU, and RMSNorm	14
5.1	System architecture and agent flow	16
5.2	Agent workflow	17
6.1	Workflow of Agent 1: Structural Profiling	19
6.2	Per-line delimiter detection and fallback in <code>_read_mixed_delimiter_csv</code> (Agent 1)	20
6.3	Agent 2 workflow (stage 1): local type inference and prompt construction	22
6.4	Agent 2 workflow (stage 2): batched LLM inference and schema parsing	22
6.5	Agent 2 workflow (stage 3): enrichment, confidence scoring, and fallback	23
6.6	Workflow of Agent 3: Preprocessing Pipeline	25
6.7	Decimal/thousands-separator resolution in <code>_normalize_currency_text</code> (Agent 3)	26
6.8	Workflow of Agent 4: Statistical Analysis	28
6.9	z-score anomaly detection in <code>_detect_anomalies</code> (Agent 4)	29

LIST OF TABLES

3.1	Current requirement coverage in the codebase	8
6.1	Implementation snapshot at the mid-point	30
A.1	Indicative project cost estimate	35
B.1	Simplified project timeline	36

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BI	Business Intelligence
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DOECE	Department of Electronics and Computer Engineering
GQA	Grouped-Query Attention
HTML	HyperText Markup Language
IOE	Institute of Engineering
JSON	JavaScript Object Notation
KV	Key-Value (cache)
LLM	Large Language Model
LPU	Language Processing Unit
PDF	Portable Document Format
RoPE	Rotary Position Embedding
TU	Tribhuvan University

1. INTRODUCTION

1.1. Background

Data-driven decision-making has become a foundational requirement for organizational effectiveness in health, agriculture, education, and commerce. In Nepal, the government's Digital Nepal Framework explicitly prioritizes data-centric governance across these sectors. However, the operational reality for most organizations particularly those outside the Kathmandu Valley remains characterized by manual spreadsheet management and underutilized datasets. The technical barrier to entry for modern analytics is high: tools demand proficiency in query languages, schema design, or statistical software, while enterprise platforms carry licensing costs that are unsustainable for donor-funded non-governmental organizations (NGOs) and municipal bodies. The emergence of LLMs and agentic orchestration frameworks has created a new design space for autonomous software systems [1]. LLMs can now interpret column semantics and narrate statistical findings in natural language, while agentic frameworks such as LangGraph enable reliable, stateful, multi-step automation. The convergence of these technologies makes it feasible to build a pipeline that replicates the workflow of a human data analyst validation, cleaning, profiling, visualization, statistical testing, interpretation, and reporting without requiring human intervention at each step.

1.2. Motivation

The motivation for this project is based on the following points:

- Despite the availability of data, many organizations lack the technical expertise required to analyze and interpret it effectively.
- Traditional data analysis tools often require statistical knowledge, programming skills, or manual report interpretation.
- Existing AI-based systems using LLMs may generate unverified or hallucinated insights, leading to unreliable conclusions.
- Recent advances in Artificial Intelligence and Multi-Agent Systems provide an opportunity to automate complex analytical tasks.

1.3. Problem Definition

Organizations accumulate data daily sales records, transaction logs, expense sheets, customer information yet cannot efficiently convert this data into insights that drive strategy and decision-making.

Existing challenges:

1. **Manual workflows:** Data analysis currently requires skilled staff to manually clean data, calculate metrics, build charts, and compile reports a process taking days to weeks
2. **Lack of standardization:** Without a systematic methodology, different people produce inconsistent results, making it difficult to compare periods or validate findings
3. **Limited accessibility:** Analytics tools require specialized knowledge. Non-technical stakeholders cannot ask ad-hoc questions without analyst intermediaries

This project proposes an end-to-end automated system that takes raw sales and financial CSV data and produces validated, auditable, professionally formatted reports with trend insights and recommendations.

1.4. Objectives

The primary objectives of this project are:

1. Develop an autonomous multi-agent system that converts raw sales and financial data (CSV format) into professional analytical reports and forecasts without manual coding
2. Democratize basic financial analytics and sales optimization for small to medium businesses and organizations through a no-code web interface

1.5. Scope and Applications

Technical Scope:

- Supports common business data types: sales records, transaction logs, expense sheets, in CSV format
- A six-agent pipeline design; four agents (profiling, semantic tagging, preprocessing, and statistical analysis) are currently implemented, while validation and report generation are planned
- Runs as web application without requiring to download additional software

Primary Use Cases:

1. **Sales Performance Tracking:** Automated analysis of sales data to identify trends, top-performing products, seasonal patterns, and revenue forecasts for planning

2. **Financial Reporting & Budgeting:** Generation of expense summaries, income vs. expenditure analysis, budget tracking, and monthly/quarterly financial reports for small to medium organizations
3. **Retail & E-commerce Analytics:** Customer purchase behavior, inventory turnover analysis, demand forecasting, and sales channel comparison
4. **Small Business Operations:** Cash flow trends, profit margin analysis, supplier/payment tracking, and operational dashboards without requiring accounting software expertise
5. **Financial Data for Non-Finance Teams:** Simplified reports that translate raw financial/sales data into plain-language insights for managers, project leads, and decision-makers

2. LITERATURE REVIEW

2.1. Traditional Data Analytics Platforms

Platforms for Business Intelligence like Microsoft Power BI, Tableau, and Google Looker Studio are popular choices when it comes to data visualization, creation of dashboards, and decision making. These technologies offer a variety of data connectivity options, visualization functions, and reporting features, helping companies track their key performance indicators and business processes. With all their ecosystem and integration possibilities, these BI systems can be used by large corporations. However, there is a number of disadvantages of BI systems as well. First of all, users need to know database schema, configure their dashboards, and prepare the data themselves. Moreover, licensing and other infrastructure expenses may prove to be expensive for small companies, educational institutions, and non-profit organizations lacking funds. However, one of the major drawbacks of traditional BI systems is their lack of automation in interpretation, relying only on visualization. According to Dwivedi et al. [1], large language models are changing data analytics. LLMs let us interact naturally, reason automatically, and get smart decision support. The authors argue that we need to progress from static dashboards to AI systems that offer insightful auto-recommendations. This points out an important research gap. To address it, we should develop intelligent, multi-agent data analyst systems.

2.2. AI-Assisted Analytics and Large Language Model-Based Data Analysis

LLMs are changing data analytics with their ability to chat naturally about data [1]. Tools like ChatGPT with Code Interpreter and other assistant-style systems let people upload datasets and get back code, charts, stats analyses, and plain text explanations [2]. This is big because it makes data analysis easier for those without a ton of technical skills.

One huge plus is how easy these platforms are to use. Regular folks can talk to a dataset without needing loads of coding knowledge. Researchers show that LLMs can handle lots of jobs, like cleaning data, doing exploratory analysis, spitting out visuals, and even putting together reports [3].

Still, there are some downsides. For one, getting the same result each time is tricky since outcomes rely heavily on the questions you ask. You might phrase things differently and end up with entirely distinct findings. Also, these systems tend to be single agent setups that don't follow a strict workflow or include checks for accuracy. There's no built-in QA process, and sometimes the code generated can have errors or missing steps [4, 5]. This really drives the need for more complex systems that employ different agents

for doing analysis, verifying results, and generating reports.

2.3. Automated Data Visualization from Tabular/CSV Data

Several studies have investigated automated visualization generation from structured datasets. Data2Vis [6] proposed a sequence-to-sequence neural architecture that translates datasets into Vega-Lite visualization specifications. VizML [7] applied machine learning to learn visualization design choices from over one million dataset-visualization pairs. Draco [8] formalized visualization design knowledge as logical constraints to recommend effective visual encodings. DeepEye [9] further automated chart generation and ranking. These studies collectively demonstrate the feasibility of automated visualization generation and provide strong theoretical support for the Visualization Agent proposed in this research.

The main insight from this line of work is that visualization creation can be treated as a recommendation or generation task rather than a purely manual design task. This is especially relevant for CSV-based analysis workflows, where the system must decide whether to use bar charts, line charts, scatter plots, or other encodings based on the structure and semantics of the tabular data. The proposed project adopts this idea by using an agent that converts cleaned tabular outputs into appropriate static visualizations for reports.

2.4. Agentic AI Frameworks and Multi-Agent Systems

The newest stuff in Agentic AI includes LangGraph, CrewAI, and AutoGen frameworks that help different AI agents work together on big projects [10, 11]. Traditionally, one AI does all the work, but these new setups let multiple agents take on tasks based on what they're best at.

LangGraph helps manage agent teamwork with things like conditional routing and checkpointing. Meanwhile, AutoGen is all about letting agents chat back and forth, and CrewAI handles task delegation based on roles. Altogether, these tools show promise for building really smart, self-reliant systems that can figure out tough problems, plan, and coordinate tasks efficiently.

According to Wu et al. [11], when you set up proper structures for AI conversations, teams of specialized agents can ace complex assignments. This covers everything from coding and math to answering questions and making decisions. It's clear that such collaborations do better than lone agents.

Still, most work right now only applies these frameworks to areas like software engineering, web searches, Q&A, and general reasoning. Not many have looked into how well they fit into fixed data analysis processes that mix deterministic stats ops with more uncertain language-model thinking. This gap presents an opportunity for the

development of multi-agent systems specifically designed for automated data analysis and business intelligence workflows.

2.5. Existing Research on Automated Data Analysis Systems

Researchers are looking into artificial intelligence to make data analytics easier. They're mixing machine learning, big language models, and natural language processing to help with exploring data, making it visual, and getting insights.

Sun et al. came up with LAMBDA, a Large Model Based Data Agent, which uses special AI agents to do data analysis [3]. These agents follow instructions in regular language to do tasks like code generation, stats, and visualizations. The cool thing is, when these agents work together, they improve accuracy and dependability more than solo agents [3].

While there's been progress, most systems now only cover certain parts of the analytics job. For instance, statistical tools are great but aren't very user-friendly, and some language model tools don't always deliver consistent results because they rely too much on how you write prompts. Also, many current tools work in isolation they don't all come together to form that full analytics process which would include everything from pre-processing data to generating reports.

So, we need a better way. There's a call for a complete multi-agent setup that provides reliable and repeatable data analytics. This could revolutionize how we handle big data tasks.

2.6. Research Gap and Project Positioning

According to the literature reviewed, there is a lack of studies dedicated to multi-agent data processing systems that combine deterministic analytical algorithms with LLM-based reasoning in a reliable end-to-end analytics pipeline [3, 11]. Traditional BI tools require significant user involvement and technical expertise [1]. While LLM-based systems improve accessibility, they do not inherently guarantee reproducibility, auditability, and structured execution workflows [4, 5]. Multi-agent frameworks provide orchestration capabilities but have primarily been applied to software engineering and general reasoning tasks rather than autonomous data analytics and reporting [11, 10]. Therefore, this project proposes a LangGraph-based multi-agent analytics system in which specialized agents perform data ingestion, preprocessing, analysis, visualization, validation, and report generation. Deterministic analytical methods will be used for statistical computations, while an LLM-based agent will generate insights and explanations. The proposed system aims to improve reproducibility, accessibility, and automation of data analytics workflows for educational institutions, small businesses, and non-technical users.

3. REQUIREMENT ANALYSIS

3.1. Functional Requirements

- Upload or load a structured dataset.
- Profile the dataset structure and missingness.
- Infer semantic metadata for columns.
- Preprocess and clean the data.
- Generate statistical summaries and visualizations.
- Validate the outputs before final reporting.
- Produce a final executive report.

3.2. Non-Functional Requirements

- Reproducible execution through shared state and deterministic processing.
- Extensible agent-based design.
- Graceful handling of missing data, invalid input, and validation failures.
- Clear audit trail for preprocessing and validation decisions.

The most important non-functional requirement is traceability. The system does not hide intermediate decisions inside a black box; instead, it keeps outputs in the shared state and logs important actions as the pipeline proceeds. A second important requirement is robustness, because a report-generating system is only useful if it can continue operating sensibly when some inputs are incomplete or imperfect.

3.3. Requirement Status

Table 3.1: Current requirement coverage in the codebase

Requirement	Status	Notes
Data loading and profiling	Implemented	Supports CSV and Excel inputs
Semantic tagging	Implemented	Heuristics plus LLM fallback
Preprocessing and cleaning	Implemented	Audit trail and quality scoring included
Statistics and charts	Implemented	Multiple chart and analysis types available
Validation gate	In progress	Agent 5 not yet implemented
Final report generation	In progress	Agent 6 not yet implemented
Frontend dashboard	Planned	Not yet the main focus
Multi-format expansion	Planned	Future support target

4. THEORETICAL APPROACH

The semantic tagging stage of this project relies on a Transformer-based Large Language Model (LLM) accessed through the Groq API, which serves the llama-3.3-70b-versatile model at high inference speed via purpose-built hardware. This chapter summarises only the concepts needed to understand how the model produces the JSON schema blueprint that drives preprocessing, and deliberately avoids theory that the implementation does not use.

4.1. Large Language Model Theory

A Large Language Model is a neural network trained to model the probability distribution of natural language: given a sequence of previous tokens, it estimates the most likely next token. In this project the model is not used for free-form prose but as a structured reasoning component that reads column metadata and returns a strict JSON schema blueprint.

4.1.1. Decoder-Only Transformer Architecture

Earlier sequence models such as RNNs and LSTMs processed tokens one step at a time, which made training slow and long-range relationships hard to capture. The Transformer replaces recurrence with a self-attention mechanism that relates every token to every other token in parallel [12]. Text-generation models such as the Llama family use a *decoder-only* Transformer: a stack of identical blocks in which a causal mask allows the prediction at position t to attend only to positions 1 to t . This left-to-right constraint makes the model autoregressive, generating one token at a time.

4.1.2. Tokenization and Embedding

Raw text is first split into subword tokens, each mapped to an integer index in a fixed vocabulary of size V . Every index is then converted into a dense vector through a learned embedding matrix $E \in \mathbb{R}^{V \times d}$, where d is the model dimension:

$$x = E(\text{token}) \tag{4.1}$$

Because self-attention is order-agnostic, a positional encoding is added to each embedding so the model can distinguish tokens by position as well as identity. In this project, the prompt containing column names, inferred types, and sample values is tokenized and embedded in exactly this way before inference.

4.1.3. Self-Attention

Self-attention lets each token gather information from other tokens. Three projections are formed from the input: queries Q , keys K , and values V , and they are combined using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (4.2)$$

where d_k is the key dimension and $\sqrt{d_k}$ scales the dot products to keep the softmax stable [12]. The Transformer runs several attention *heads* in parallel so the model can capture different relationships at once. Each block also applies a position-wise feed-forward network with residual connections and layer normalization, which add capacity and keep the deep computation numerically stable.

4.1.4. Next-Token Prediction

After the final block, the last token representation is projected to a vector of logits z of length V , which a softmax converts into a probability distribution over the vocabulary:

$$P(y_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (4.3)$$

The model predicts the next token conditioned on all previous tokens and generates text autoregressively: each predicted token is appended to the sequence and fed back in to predict the following one, until a stopping condition is reached. A low sampling temperature is used in this project so the output is nearly deterministic, which is important because the response must be strictly valid JSON.

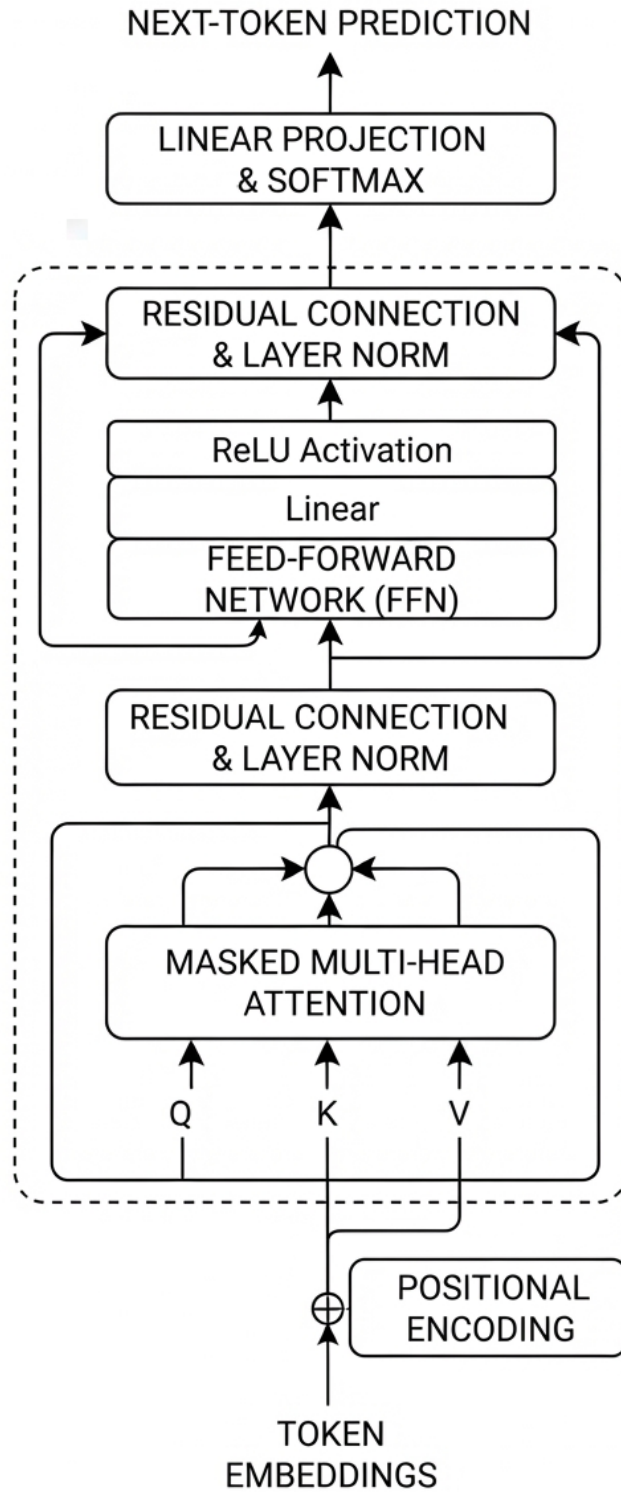


Figure 4.1: Decoder-only Transformer architecture used by modern LLMs

4.2. Groq API and LLM Inference Architecture

The LLM backend of this system is the Groq API, which exposes the llama-3.3-70b-versatile model through an OpenAI-compatible REST interface. Groq achieves exceptionally low inference latency by running Transformer models on

Language Processing Units (LPUs), custom silicon designed specifically for autoregressive token generation. This section explains the Groq inference model and how Agent 2 uses it to generate the JSON schema blueprint.

4.2.1. Why the Groq API Is Used

The Groq API was selected over local inference runtimes for three principal reasons.

Inference speed. LPU hardware is architected for the sequential, memory-bound nature of autoregressive decoding. Each forward pass generates one token, and the dominant cost is reading model weights from memory on every step. LPUs use on-chip SRAM instead of off-chip DRAM, eliminating memory bandwidth as the bottleneck. In practice, `llama-3.3-70b-versatile` on Groq produces hundreds of output tokens per second—far more than a CPU or consumer GPU running the same model weights locally. For a pipeline where Agent 2 may need to tag dozens of columns in one call, short round-trip times keep the overall pipeline latency acceptable.

No local hardware requirement. Running a 70-billion-parameter model locally demands tens of gigabytes of GPU VRAM. The Groq API allows the project to use a state-of-the-art open-weight model without imposing any GPU requirement on the developer’s machine. This makes the system deployable on standard laptops and cloud VMs alike.

Simplicity and cost control. The API follows the same chat-completion contract as the OpenAI SDK. Agent 2 constructs a system prompt and a user message, sends a single HTTP POST request, and receives a JSON response. The official Python client (`groq`) abstracts authentication and retries. Usage-based pricing means the project incurs no fixed infrastructure cost, and because Agent 2 sends only column metadata and three sample values rather than full table rows, prompt token counts remain small.

4.2.2. The Llama 3.3 70B Model

`llama-3.3-70b-versatile` is a 70-billion-parameter decoder-only Transformer released by Meta as part of the Llama 3 family [13]. Its architecture follows the design described in Section 4.1 with several refinements that improve efficiency and long-context handling.

Grouped-Query Attention (GQA). Standard multi-head attention stores one key–value head per query head. Grouped-Query Attention partitions the query heads into groups that share a single key–value head. Formally, if there are H query heads organized into G groups ($G < H$), the attention for query head h in group g is:

$$\text{GQA}_h(Q_h, K_g, V_g) = \text{Softmax}\left(\frac{Q_h K_g^\top}{\sqrt{d_k}}\right) V_g \quad (4.4)$$

where K_g and V_g are the shared key and value matrices for group g [14]. GQA reduces the size of the KV cache proportionally to the group ratio H/G , which lowers memory bandwidth pressure during autoregressive decoding and is a key reason LPU-based inference is particularly effective with this model.

RoPE positional encoding. Instead of adding a fixed positional vector to each embedding, Llama 3 applies Rotary Position Embedding (RoPE), which encodes absolute position by rotating the query and key vectors before the dot product:

$$\tilde{q}_m = q_m \cdot e^{im\theta}, \quad \tilde{k}_n = k_n \cdot e^{in\theta} \quad (4.5)$$

After rotation the dot product $\tilde{q}_m^\top \tilde{k}_n$ depends only on the relative position $m - n$, giving the model a principled notion of token distance [15]. Compared with learned absolute embeddings, RoPE generalizes more cleanly to prompt lengths not seen during training.

SwiGLU feed-forward blocks. Each Transformer block contains a position-wise feed-forward network. Llama 3 replaces the standard ReLU activation with SwiGLU:

$$\text{FFN}(x) = (\text{Swish}(xW_1) \odot xW_2) W_3 \quad (4.6)$$

where $\text{Swish}(x) = x \cdot \sigma(x)$ is a smooth gating function, $\odot$ is element-wise multiplication, and W_1, W_2, W_3 are learned weight matrices [16]. The gating mechanism allows the network to suppress irrelevant dimensions adaptively, which empirically improves output quality over ReLU at the same parameter count.

RMS Layer Normalization. Llama 3 uses Root Mean Square Layer Normalization (RMSNorm) instead of the standard LayerNorm:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \cdot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \quad (4.7)$$

RMSNorm omits the mean-centering step of LayerNorm [17]. Because re-centering is often redundant given downstream weight matrices, RMSNorm achieves similar training stability at lower computational cost.

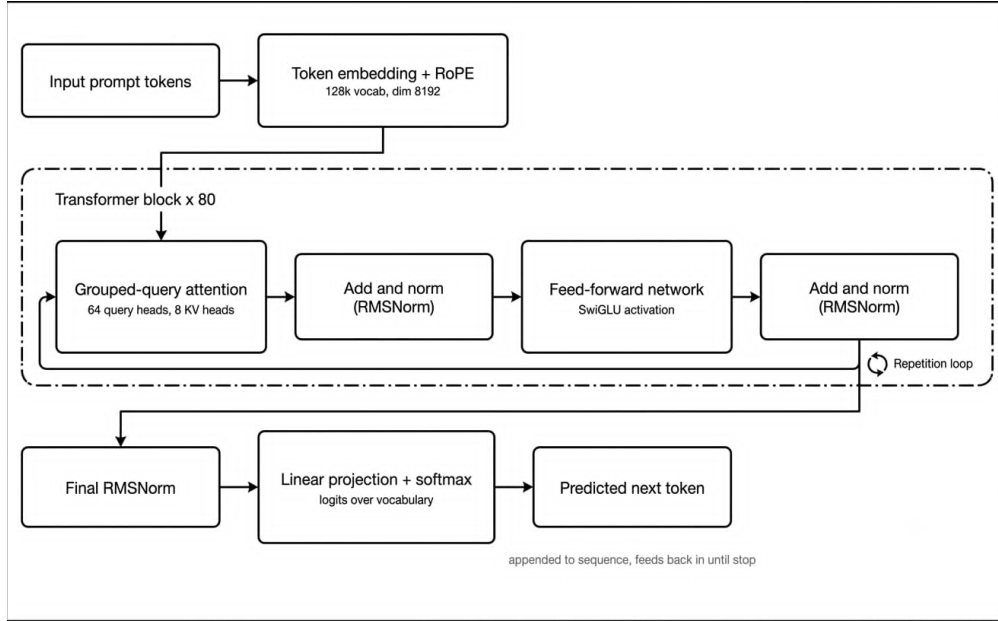


Figure 4.2: Llama 3.3-70B architecture: decoder-only Transformer with GQA, RoPE, SwiGLU, and RMSNorm

4.2.3. Agent 2 API Call and Prompt Processing

Agent 2 calls the Groq API exactly once per tagging request (or per batch for wide schemas). The call flow is as follows.

1. **Prompt construction.** `_build_llm_prompt` assembles a *system* message that specifies the required JSON schema format and valid tag values, and a *user* message containing, for each column: the column name, the heuristically inferred type, the missing-value rate, and the first three sample values. This minimal context keeps the prompt under a few hundred tokens regardless of table size.
2. **API request.** The Groq Python client sends a `chat.completions.create` request with `model="llama-3.3-70b-versatile"` and `temperature=0.1`. A temperature close to zero makes the softmax distribution (Equation 4.3) sharply peaked, so the model consistently selects the highest-probability token rather than sampling freely. This near-determinism is important because the output must be strictly parseable JSON.
3. **Prompt processing.** On Groq's LPU cluster the prompt tokens are tokenized and embedded (Equation 4.1), then passed through the 80 decoder blocks of Llama 3.3-70B. Inside each block, GQA (Equation 4.4) computes position-aware attention using RoPE (Equation 4.5), and the SwiGLU FFN (Equation 4.6) applies non-linear gating. RMSNorm (Equation 4.7) stabilizes each sub-layer. The final block projects to logits over the vocabulary and applies softmax to produce the next-token distribution.

4. **KV cache and autoregressive generation.** After the prompt is processed, the key and value tensors for every prompt token are cached. Each subsequent output token is generated by running only the new token through the network, attending to the cached values. This KV-cache mechanism makes generation fast even for long prompts.
5. **Response parsing.** The generated text is returned in the API response body. `_parse_schema_blueprint_response` extracts valid JSON from raw or code-fenced output; as a last resort it locates the outermost `{ . . . }` span so that a stray formatting character does not discard an otherwise correct schema. If parsing fails after retries, `_fallback_blueprint` constructs a conservative schema from the locally sniffed types, ensuring the pipeline always has a usable blueprint.

This design keeps Agent 2 loosely coupled to the model provider: the entire LLM interaction is isolated inside three functions (`_build_llm_prompt`, the Groq client call, and `_parse_schema_blueprint_response`), so the backend can be swapped for another OpenAI-compatible endpoint without touching the rest of the pipeline.

5. SYSTEM ARCHITECTURE AND METHODOLOGY

5.1. Overall Pipeline

The backend pipeline is built as a directed sequence of agents. Each agent consumes the current GraphState, writes its own outputs back into the shared state, and passes the result to the next step. This keeps the workflow clear and makes debugging easier.

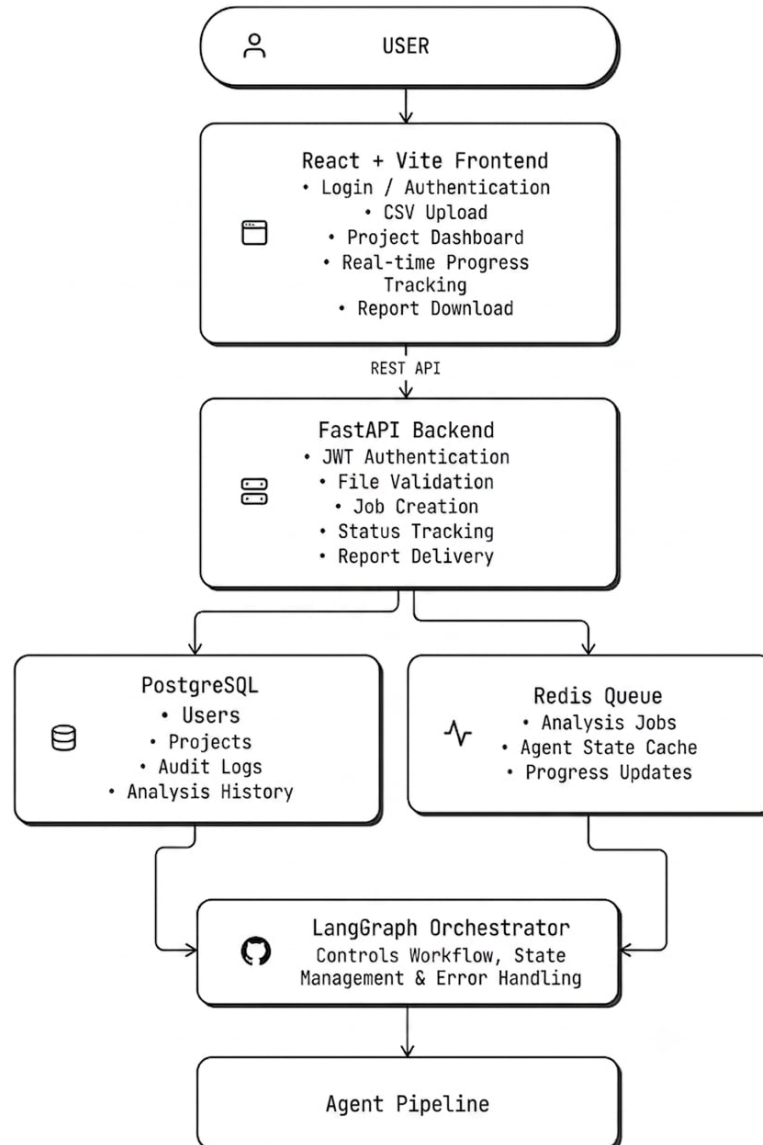


Figure 5.1: System architecture and agent flow

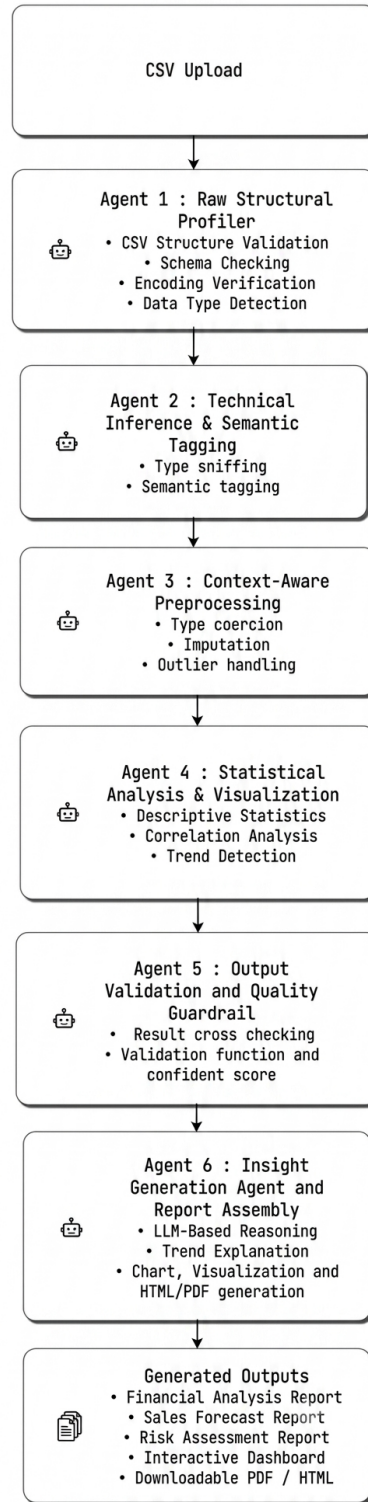


Figure 5.2: Agent workflow

5.2. Shared State Design

The shared state contains the input path, the raw profile, the semantic blueprint, the cleaned dataset, the scaling parameters, the preprocessing log, the quality score, the statistics, the chart paths, an error list, and reliability metadata. This design allows each

agent to focus on one responsibility while still keeping the whole workflow connected. Fields for the validation result and the final report path will be added to the state once Agent 5 and Agent 6 are implemented.

The state also carries reliability metadata. Each stage can contribute a confidence score and evidence list, which helps the system decide whether a later stage should proceed. That is useful in the report because it shows that the project is not just generating outputs, but also measuring the reliability of those outputs.

5.3. Methodology

The methodology combines deterministic data processing with LLM-assisted semantic reasoning. Structural checks, preprocessing rules, and statistics are implemented in Python, while the validation and reporting stages remain planned work. Semantic interpretation uses the Groq API with the llama-3.3-70b-versatile model, and the planned final narrative generation will use the same backend; both stages are backed by fallback logic so the pipeline remains usable when the API is unavailable.

Methodologically, the project follows a layered analytics model. The early stages reduce raw data into a trustworthy form, the middle stages compute measurable patterns, and the later stages turn those patterns into human-readable findings. That mirrors the workflow of a human analyst while still keeping the process automated and repeatable.

The implementation approach is also incremental. Each agent was developed as a distinct module, which made it easier to test one stage at a time and identify integration issues early. This is a good fit for a mid-term report because it shows progress in the structure of the system, not only in the final output.

6. IMPLEMENTATION DETAILS

6.1. Agent 1: Structural Profiler

6.1.1. Overview and Purpose

Agent 1 (`agent1_structural_profiler`) is the entry point of the pipeline. It reads the input file and records a factual description of its structure, performing no cleaning or inference so that every later stage works from an unmodified baseline.

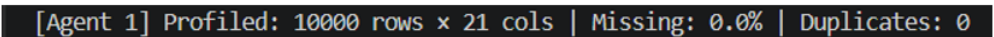
6.1.2. Inputs and Outputs

- **Inputs:** `csv_path` from the shared `GraphState`, an optional `sheet_name` for Excel workbooks, and the running errors list.
- **Outputs:** `raw_profile` (per-column and dataset-level summary), the cached `DataFrame _df_cache`, updated errors, and stage reliability metadata.

6.1.3. Processing Workflow

The agent proceeds in four ordered stages, summarised in Figure 6.1:

1. `_load_dataframe` selects a reader based on the file extension, routing Excel and CSV files to dedicated loaders.
2. the chosen reader tolerates real-world messiness such as mixed delimiters and non-UTF encodings.
3. each column is measured for type, missingness, uniqueness, and sample values.
4. a confidence value is computed from the observed missingness and duplication.



```
[Agent 1] Profiled: 10000 rows x 21 cols | Missing: 0.0% | Duplicates: 0
```

Figure 6.1: Workflow of Agent 1: Structural Profiling

6.1.4. Key Functions

`_read_mixed_delimiter_csv`: **resilient CSV ingestion.** Business exports are often inconsistent, mixing comma- and semicolon-separated rows under non-UTF encodings. This function first reads the text through `_read_csv_lines` (trying `utf-8-sig`, `cp1252`, and `latin-1`), then detects the delimiter *per line* and, when the field count does not match the header, retries with the alternate delimiter before raising an error (Figure 6.2). It matters because a wrong delimiter would silently merge columns and corrupt every

later stage. The column profiling that follows records each column's dtype, missing count and rate, unique count, and first three sample values, which Agent 2 later uses instead of the full dataset.

```

1 for line in lines[1:]:
2     if not line.strip():
3         continue
4
5     delimiter = ";" if line.count(";") > line.count(",") else
6         ","
7     fields = next(csv.reader([line], delimiter=delimiter,
8         quotechar='"'))
9
10    if len(fields) != len(header):
11        alternate = "," if delimiter == ";" else ";"
12        alt_fields = next(csv.reader([line],
13            delimiter=alternate, quotechar='"'))
14        if len(alt_fields) == len(header):
15            fields = alt_fields
16        else:
17            raise ValueError(
18                f"Unable to parse row with {len(fields)}
19                fields "
20                f"(expected {len(header)}): {line[:120]}")
21
22    writer.writerow(fields)

```

Figure 6.2: Per-line delimiter detection and fallback in `_read_mixed_delimiter_csv` (Agent 1)

6.1.5. Reliability and Pipeline Interaction

`_compute_agent1_confidence` converts the observed data health into a single confidence value that decreases with missingness and duplication:

$$c_1 = \text{clamp}\left(1 - 0.5 \frac{m}{100} - 0.3 \frac{d}{100}, 0, 1\right) \quad (6.1)$$

where m is the overall missing rate and d the duplicate rate as percentages. The stage is marked ready when $c_1 \geq 0.8$ and `needs_review` otherwise. On success it writes `raw_profile`, caches the DataFrame in `_df_cache`, and passes both to Agent 2; if loading fails, the error is recorded and the pipeline stops early. Tested on the Amazon Sales Dataset, Agent 1 parsed the file without row loss and produced the per-column missingness and duplicate counts consumed by the later agents.

6.2. Agent 2: Semantic Tagger

6.2.1. Overview and Purpose

Agent 2 (`agent2_semantic_tagger`) turns the structural profile into a `schema_blueprint`: a per-column description of the intended type, semantic role, and cleaning policy. Where Agent 1 only observed the data, Agent 2 assigns meaning to each column so that preprocessing can be driven by rules rather than guesswork. When the language model is invoked, it is called through the Groq API (`llama-3.3-70b-versatile`); the inference mechanics are described in Chapter 4 and are not repeated here.

6.2.2. Inputs and Outputs

- **Inputs:** `raw_profile` and the cached `DataFrame _df_cache` from Agent 1, plus the errors list.
- **Outputs:** the `schema_blueprint` (per-column type, semantic tag, identifier flag, scaling and imputation policy, encoding strategy, and null policy) with a `__metadata__` block and updated errors.

6.2.3. Processing Workflow

The agent combines cheap local heuristics with a controlled model call. The workflow is illustrated in three stages in Figures 6.3 to 6.5.

First, local type sniffing (`_infer_intended_types`) assigns a provisional type to every column, and a minimal prompt containing only column metadata and three sample values is built (Figure 6.3).

```
[Agent 2] Type sniffing summary: {'unknown': 16, 'numeric': 5}
[Agent 2] Blueprint built for 21 columns
[Agent 2] Semantic tags by column:
```

```
└─ order_id
   semantic_tag      : identifier
   intended_type     : string
   imputation        : drop
   encoding          : none
   identifier        : True
   analysis_allowed  : True
   confidence        : 70.0
   null_action       : drop_rows
   suitable          : True
   reason            : identifier_clean

└─ order_date
   semantic_tag      : datetime
   intended_type     : datetime
   imputation        : none
   encoding          : none
   identifier        : False
   analysis_allowed  : True
   confidence        : 60.0
   null_action       : none
   suitable          : True
   reason            : datetime_missingness
```

Figure 6.3: Agent 2 workflow (stage 1): local type inference and prompt construction

Next, the model is called once for small schemas or in retry-safe batches for wide datasets and its JSON response is parsed defensively into the schema blueprint (Figure 6.4).

```
└─ country
   semantic_tag      : geographic
   intended_type     : string
   imputation        : mode
   encoding          : one_hot
   identifier        : False
   analysis_allowed  : True
   confidence        : 60.0
   null_action       : impute_mode
   suitable          : True
   reason            : low_cardinality_category
```

Figure 6.4: Agent 2 workflow (stage 2): batched LLM inference and schema parsing

Finally, the blueprint is enriched: missing tags are filled, a missingness policy is applied, per-column confidence is scored, and a conservative fallback is used if the model output is unusable (Figure 6.5).

```
unit_price
semantic_tag      : currency
intended_type     : float
imputation        : flag_only
encoding          : none
identifier        : False
analysis_allowed  : True
confidence        : 85.0
null_action       : flag_only
suitable          : True
reason            : numeric_missingness
```

Figure 6.5: Agent 2 workflow (stage 3): enrichment, confidence scoring, and fallback

6.2.4. Key Functions

`_infer_intended_types` performs local type sniffing without the model, using the Pandas dtype and an 80% parseability threshold for object columns; its result seeds both the prompt and the fallback, so every column has a defensible type even when the LLM is unavailable. `_build_llm_prompt` sends only column metadata and three samples rather than the full table, and a fixed system prompt requires strictly valid JSON at a low temperature (0.1). For wide schemas, `_call_llm_for_schema_blueprint_with_retry` splits columns into batches and, if a truncated response fails to parse, recursively halves the batch and merges the results, which lets the agent tag very wide datasets reliably. `_parse_schema_blueprint_response` accepts raw or fenced JSON and, as a last resort, extracts the outermost `{...}` span, so a single formatting quirk does not discard a valid schema.

6.2.5. Reliability and Pipeline Interaction

After tagging, missing `semantic_tag` values are filled and each column receives a `null_policy` and `encoding_strategy`; columns whose missing rate exceeds 20% are marked `analysis_allowed = false` so later analysis ignores unreliable fields, and `_calculate_semantic_confidence` scores each tag from 0 to 100. If the model output is invalid, `_fallback_blueprint` assigns conservative policies from the sniffed types. On the Amazon Sales Dataset, Agent 2 tagged the price and discount columns as cur-

rency and the rating and quantity columns as numeric, while flagging high-missingness columns for exclusion from analysis. The resulting `schema_blueprint` is the contract that drives every decision in Agent 3.

6.3. Agent 3: Preprocessor

6.3.1. Overview and Purpose

Agent 3 (`agent3_preprocessor`) performs deterministic cleaning and feature construction, guided entirely by the `schema_blueprint` from Agent 2. It is the stage that turns raw, inconsistent values into a trustworthy, analysis-ready table. Unlike Agent 2, it uses no language model at all: every transformation is a fixed, logged rule, which makes the output reproducible and auditable.

6.3.2. Inputs and Outputs

- **Inputs:** the cached `DataFrame` `_df_cache`, the `schema_blueprint`, and `raw_profile`. The agent returns early if either the frame or the blueprint is missing.
- **Outputs:** `cleaned_df`, `cleaned_csv_path`, `scaling_params`, `preprocessing_log`, `data_quality`, a `column_ledger`, and reliability metadata.

6.3.3. Profile Selection

Before any cleaning, `_build_preprocessing_config` chooses a `strict`, `balanced`, or `lenient` configuration profile. A profile pinned by the caller is always honoured; otherwise `_detect_dataset_domain` inspects finance keywords and currency/percentage tags and selects `strict` for finance and sales data. Each profile fixes the currency limits, the maximum plausible tax rate, the reconciliation tolerances, and the weights used later in the quality score. This step matters because it lets a single code base apply appropriately cautious rules to sensitive financial data while staying flexible on generic datasets.

6.3.4. Processing Workflow

Cleaning runs as an ordered, fully logged sequence of steps; each step records its per-column null differences so the effect of every action is visible. The stages, shown in Figure 6.6, are:

1. **Step 0: Deduplication** `dedup_exact_rows` removes exact duplicate rows, asserts the count matches Agent 1's profile, and aborts through

`_assert_row_survival_or_abort` if fewer than half the rows survive (a safeguard against accidental data loss).

2. **Step 1: Currency cleaning** `_clean_currency_values` normalises monetary text and writes a `_parse_failed` flag for values it cannot parse.
3. **Step 2: Type coercion** `_coerce_types` casts each column to its intended type while tracking any new NaNs created by the cast.
4. **Step 3/3b: Text standardisation and re-dedup** text columns are canonicalised, then duplicates are removed again in case canonicalisation merged rows.
5. **Step 4: Imputation** `_impute` fills or drops missing values according to each column's policy.
6. **Step 5: Encoding** categorical columns are encoded for later analysis.
7. **Step 6: Outlier clipping** `_clip_outliers` caps extreme values using percentile bounds for skewed columns and the IQR $1.5\times$ rule otherwise.
8. **Step 7: Scaling** `_scale_columns` applies Min-Max scaling while preserving raw copies.
9. **Step 8: Date features** `_extract_date_features` derives calendar features from datetime columns.
10. **Step 9: Business metrics** `_derive_business_metrics` constructs profit, margin, and related fields where the necessary columns exist.

```
[Agent 3] Preprocessing: profile=strict domain=finance_sales input=10000x21
[Agent 3] Completed: rows=10000->10000 cols=21->115 quality=100.0/100 raw_missing=0.0% remaining_nulls=0.0%
[Agent 3] Cleaning summary: deduped=0, scaled=1, added_cols=94, removed_rows=0
[Agent 3] Note: the cleaned dataset is normalized and exported to outputs/cleaned_data.csv; if the row count stays the same
[Agent 4] Starting analysis: 10000 rows x 115 cols
```

Figure 6.6: Workflow of Agent 3: Preprocessing Pipeline

6.3.5. Key Functions

`_normalize_currency_text`: **monetary parsing.** Financial exports mix symbols, currency codes, parenthetical negatives such as (123), and locale-specific separators (for example the European 1.234,56 versus the US 1,234.56). This function converts all of these to a plain numeric string so the values can be cast to `float`. Its most important part, shown in Figure 6.7, decides which separator is the decimal point by comparing the positions of the last comma and last dot, then strips the thousands separator. Without this step, monetary columns would be read as text and every downstream financial metric would be impossible.

```

1 last_comma = text.rfind(",")
2 last_dot = text.rfind(".")
3
4 if last_comma != -1 and last_dot != -1:
5     decimal_sep = "," if last_comma > last_dot else "."
6     thousands_sep = "." if decimal_sep == "," else ","
7     text = text.replace(thousands_sep, "")
8     text = text.replace(decimal_sep, ".")
9 elif last_comma != -1:
10    tail = text[last_comma + 1:]
11    if re.fullmatch(r"\d{2}", tail):          # 1234,56 ->
12        1234.56
13        text = text[:last_comma].replace(",", "") + "." + tail
14    else:                                     # 1,234 -> 1234
15        (thousands)
16        text = text.replace(",", "")

```

Figure 6.7: Decimal/thousands-separator resolution in `_normalize_currency_text` (Agent 3)

Other key functions. `_impute` fills or drops missing values according to each column’s `null_policy` (mean, median, mode, unknown-label, flag-only, or identifier row-drop), recording the exact value used for each column. `_scale_columns` applies Min-Max normalisation,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (6.2)$$

while keeping `_raw` and `_scaled` copies and storing $x_{\min}$ and $x_{\max}$ in `scaling_params`, so the transform can be inverted later. `_derive_business_metrics` uses whole-word matching (`_find_col`, which avoids false hits such as “count” matching “Country”) to locate revenue, cost, unit, price, discount, and budget columns, then derives profit, margin, revenue per unit, discount percentage, and budget variance wherever the required columns exist, giving Agent 4 richer fields to summarise.

6.3.6. Validation and Data Quality

After the transformation steps, `_validate_count_ranges` and `_validate_financial_constraints` flag invalid counts and implausible tax rates into a `ColumnLedger`, and `_compute_enhanced_quality_score` combines the remaining null rate, the validation-failure rate, and the duplication rate into a single 0 to 100 `data_quality` score. The ledger and score make the cleaning process transparent: a reviewer can see not only what the data became, but how much of it had to be corrected.

6.3.7. Reliability and Pipeline Interaction

Every step appends to `preprocessing_log` with its null differences, and the cleaned frame is exported to `outputs/cleaned_data.csv`. Following the reliability model introduced in Equation 6.1, the agent combines the quality score with the row-survival ratio into a confidence value and marks the stage `ready` or `needs_review`. Critical failures trigger `_early_exit_with_error` so a corrupted result never propagates. On the Amazon Sales Dataset, Agent 3 parsed the price and discount fields into numeric values, imputed the missing ratings, and derived discount-percentage and margin columns, producing a cleaned table with a high data-quality score. The `cleaned_df` and `scaling_params` are the direct inputs to Agent 4.

6.4. Agent 4: Statistical Analysis

6.4.1. Overview and Purpose

Agent 4 (`agent4_analysis`) turns the cleaned dataset into descriptive statistics and chart artifacts. It is the stage that produces measurable evidence about the data: summary statistics, correlations, trends, rankings, seasonality, anomalies, and regression fits, each accompanied by a saved figure. Before running, it clears previous PNGs with `_clear_chart_dir` and writes every figure through `_save` into `outputs/charts`, so each run starts from a clean, reproducible set of outputs.

6.4.2. Inputs and Outputs

- **Inputs:** the `cleaned_df` and `scaling_params` from Agent 3 and the `schema_blueprint`; it errors out if no cleaned frame exists.
- **Outputs:** a `stats` dictionary holding all numerical results, a `chart_paths` list of saved figures, updated errors, and reliability metadata.

6.4.3. Processing Workflow

The agent runs a fixed battery of analyses over the selected columns, illustrated in Figure 6.8:

1. **Column selection** restricts analysis to meaningful fields.
2. **Descriptive statistics** and a **correlation** matrix summarise each column and the relationships between them.
3. **Growth rates**, **rankings**, and **seasonality** describe how key measures change over time and across categories.

4. **Anomaly detection** and **regression trends** highlight unusual values and directional change.
5. **Distribution and derived-metric charts** visualise spread and the business fields created in Agent 3.

```
[Agent 4] Starting analysis: 10000 rows x 115 cols
[Agent 4] Step 1 – Descriptive stats: 27 columns
[Agent 4] Step 2 – Correlation done, strong pairs: 8
[Agent 4] Step 3 – Growth rates skipped (no revenue/time axis)
[Agent 4] Step 4 – Rankings skipped (no revenue/category pairing)
[Agent 4] Step 5 – Seasonality skipped (no time axis)
[Agent 4] Step 6 – Anomalies: 235 unique rows (2.35%) across 1 column
[Agent 4] Step 7 – Distributions done (4 charts)
[Agent 4] Step 8 – Regression done (27 columns, 0 charts)
[Agent 4] Step 9 – Distribution charts done (1 charts)
[Agent 4] Step 10 – Derived metrics skipped (no derived metrics)
[Agent 4] Done – 6 charts saved to outputs/charts/
```

Figure 6.8: Workflow of Agent 4: Statistical Analysis

6.4.4. Key Functions

Column selection. `_numeric_cols` and `_categorical_cols` decide which columns are worth analysing, excluding validation columns (`_parse_failed`, `_range_failed`), scaling backups (`_raw`, `_scaled`), identifiers, datetime columns, and any column marked `analysis_allowed = false`. They run at the start of almost every other function, preventing meaningless output such as correlating an ID column.

Summarisation. `_descriptive_stats` computes count, mean, median, standard deviation, variance, quartiles, skewness, and kurtosis for each numeric column, while `_correlation` computes Pearson and Spearman matrices, renders a heatmap, and records the “strong” pairs ($|r| \geq 0.5$, labelled strong when $|r| \geq 0.7$), which points the reporting stage at the relationships most worth explaining.

`_detect_anomalies`: **statistical outlier flagging.** This function flags values whose absolute z -score exceeds a threshold of 3.5, using the standard score

$$z = \frac{x - \mu}{\sigma} \quad (6.3)$$

where x is a value and μ , σ are the column mean and standard deviation. It reports both the per-column anomalies and the count of unique affected rows, which lets the pipeline judge how “noisy” a dataset is. Columns with too few points or zero variance are skipped so the score stays well defined. The core loop is shown in Figure 6.9.

```

1 for col in _numeric_cols(df, schema_blueprint):
2     s = df[col].dropna()
3     if len(s) < 4:
4         continue
5     mean, std = s.mean(), s.std()
6     if std == 0:
7         continue
8     z_scores = (df[col] - mean) / std
9     anomaly_mask = z_scores.abs() > z_threshold
10    anomaly_indices = df.index[anomaly_mask].tolist()
11    if anomaly_indices:
12        all_anomaly_indices.update(anomaly_indices)

```

Figure 6.9: z -score anomaly detection in `_detect_anomalies` (Agent 4)

Trend and regression analysis. `_growth_rates` uses a revenue column together with the `_month/_year` features from Agent 3 to plot month- and quarter-over-quarter change, `_top_bottom_rankings` ranks revenue across up to three categories, and `_seasonality` plots monthly averages with the best and worst months. `_regression_trends` fits an ordinary least-squares line on the `_month` axis with `scipy.stats.linregress`, recording slope, intercept, R^2 , p -value, and whether the trend is significant ($p < 0.05$). Together these functions convert the cleaned table into the directional insights a human analyst would look for.

6.4.5. Reliability and Pipeline Interaction

The agent writes the `stats` dictionary and `chart_paths`, updates errors, and records reliability (0.9 when statistics were produced, otherwise 0.4) using the same `update_reliability` mechanism described for Agent 1. On the Amazon Sales Dataset, Agent 4 produced descriptive statistics for the price and rating columns, a correlation heatmap that showed a strong positive relationship between the actual and discounted price, and category-distribution and derived-metric charts. These statistics and charts form the evidence base that the planned validation and reporting stages (Agents 5 and 6) will consume.

6.5. Current Implementation Snapshot

Table 6.1: Implementation snapshot at the mid-point

Module	Current state	Remarks
Graph state and routing	Complete	Conditional pipeline is in place
Agent 1 to Agent 3	Complete	Profiling, tagging, and pre-processing are working
Agent 4	Complete	Charts and statistics are generated
Agent 5	In progress	Validation gate not yet implemented
Agent 6	In progress	HTML report generation not yet implemented
UI and deployment polish	In progress	Still needs refinement
Test expansion	In progress	More edge cases are being added

7. RESULTS AND ANALYSIS

7.1. Observed Progress

The current codebase already demonstrates an end-to-end pipeline from raw data ingestion through statistical analysis and chart generation. Running the pipeline on the Amazon Sales Dataset, Agent 4 generates multiple chart types after profiling, semantic tagging, and preprocessing complete. That means the system is no longer just a design proposal; it is now a working backend prototype up to Agent 4.

That progress is important because it confirms the proposal is executable rather than purely conceptual. The pipeline is already performing the core work the proposal promised: profiling, semantic tagging, preprocessing, and statistical analysis. The remaining validation and reporting stages are still to be built, which is appropriate for a mid-term checkpoint.

7.2. Practical Output

The most visible outputs produced by the current implementation are:

- a structural profile of the uploaded dataset,
- a semantic schema blueprint,
- a cleaned dataset with quality metrics and an audit trail,
- statistical summaries and charts,
- per-stage reliability metadata with confidence scores.

These outputs are useful because they show both machine-readable state and human-readable artifacts. The system can therefore be inspected by later agents as well as by project evaluators. That dual nature is one of the most practical strengths of the current implementation.

7.3. Analysis

The strongest result so far is that the system separates concerns well. Agent 1 does not try to clean data, Agent 3 does not try to infer semantics, and Agent 4 focuses only on analysis rather than cleaning. That separation makes the pipeline easier to maintain and easier to explain in a report. Another important result is the reliability mechanism: the system keeps track of confidence and decision readiness instead of assuming every stage succeeds equally well.

From an engineering perspective, the current codebase is already behaving like a real analytics pipeline. It does not just transform rows and columns; it builds a story about the dataset in stages. It starts with raw structure, adds meaning, cleans the data, and computes evidence in the form of statistics and charts. The later stages that check that evidence and produce a report are planned as the next steps. That staged sequence is the core technical value of the project.

The depth of the implementation is also visible in the way the backend handles data quality. Rather than relying on one generic cleaning rule, the pipeline uses multiple checks and logs to capture what changed and why. That makes the output more defensible and more appropriate for business datasets where context matters.

7.4. Current Challenges

The main remaining work spans both new functionality and refinement. The two unimplemented agents (validation and reporting) still need to be built, and the pipeline also needs broader testing on different dataset shapes and a clearer evaluation section for the final report. The planned validation gate will need careful tuning so that it remains strict without rejecting acceptable outputs unnecessarily.

Another challenge is balancing flexibility and consistency. The system needs to handle varied datasets, but it also needs to produce stable and repeatable results. That balance is important because the final project should be practical without becoming unpredictable.

At this stage, the project is best described as functional but still maturing. The core behavior is already present, but the last phase needs stronger evaluation, documentation, and presentation work before submission.

8. REMAINING TASKS

1. Implement Agent 5 (Validation and Quality Guardrail) to check the statistical outputs, charts, and quality metrics and to gate low-confidence results before reporting.
2. Implement Agent 6 (Report Generation) to assemble the validated statistics and charts into a self-contained final report.
3. Improve the presentation layer and decide whether a user-facing interface should be included in the final deliverable.
4. Expand automated tests across preprocessing and the new validation and reporting stages.
5. Prepare the final evaluation section with sample outputs and documented observations, then clean up the report language, citations, and formatting before final submission.

In addition to these tasks, the team needs to keep the report aligned with the actual backend behavior. As the codebase evolves, the document should continue to describe what the system really does rather than only what was originally planned. That alignment will matter in the final defense because it keeps the report credible and technically consistent.

9. CONCLUSION

The project has reached a strong mid-point. The core backend pipeline is implemented through Agent 4, the agents are coordinated through shared state, and the system can process data through profiling, semantic tagging, preprocessing, and statistical analysis. Validation (Agent 5) and report generation (Agent 6) remain to be implemented, so the remaining work combines this new functionality with refinement of the existing stages. This gives the project a solid base for final evaluation and presentation.

At this stage, the project can already be described as a functioning multi-agent analytics backend up to the statistical analysis stage. Its main value lies in combining deterministic processing with controlled language-model assistance while keeping the workflow auditable. The next phase is to build the validation and reporting agents and to strengthen the presentation, test coverage, and evaluation so that the final report can present the system confidently and with evidence.

A. COST ESTIMATE

Table A.1: Indicative project cost estimate

Item	Estimate	Notes
Development laptop and local machine	Existing	Already available for development
Python, Pandas, LangGraph, plotting stack	Existing	Open-source software stack
LLM API usage	Variable	Depends on testing and report generation
Internet and miscellaneous expenses	Variable	Project maintenance cost

The cost estimate is intentionally lightweight because the project uses open-source libraries and an existing development environment. The main variable cost comes from API usage during testing and report generation. In practice, that cost can be managed by keeping the heuristic fallback path available and by avoiding unnecessary repeated model calls.

B. PROJECT TIMELINE

Table B.1: Simplified project timeline

Task	Status	Phase
Requirement analysis and proposal	Complete	Initial
Agent implementation and integration	Mostly complete	Development
Validation and reporting refinement	In progress	Development
Testing and evaluation	In progress	Finalization
Report submission preparation	Pending	Final

This timeline reflects the actual progression of the work. The first phase focused on design and planning, the current phase focuses on integration and documentation, and the final phase will focus on evaluation and submission readiness. It provides a concise view of how the project has moved from proposal to implementation.

REFERENCES

- [1] Y. K. Dwivedi, L. Hughes, A. M. Baabdullah, et al., “So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy,” *International Journal of Information Management*, vol. 71, 2023.
- [2] OpenAI, “GPT-4 Technical Report,” 2024.
- [3] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, “LAMBDA: A Large Model Based Data Agent,” arXiv preprint arXiv:2407.17535, 2024.
- [4] Y. K. Liu, et al., “Summary of ChatGPT-Related Research and Perspective Towards the Future of Large Language Models,” arXiv:2306.07209, 2023.
- [5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” in *Proceedings of ACL*, 2020.
- [6] V. Dibia and C. Demiralp, “Data2Vis: Automatic Generation of Data Visualizations Using Sequence-to-Sequence Recurrent Neural Networks,” *IEEE Computer Graphics and Applications*, vol. 39, no. 5, pp. 33–46, 2019.
- [7] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. A. Hidalgo, “VizML: A Machine Learning Approach to Visualization Recommendation,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2019.
- [8] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer, “Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco,” *IEEE Transactions on Visualization and Computer Graphics*, 2018.
- [9] Y. Luo, X. Qin, N. Tang, and G. Li, “DeepEye: Towards Automatic Data Visualization,” in *IEEE International Conference on Data Engineering (ICDE)*, 2018.
- [10] LangChain, “LangGraph: A Python Library for Building Stateful, Multi-Agent Applications,” 2024.
- [11] Q. Wu, G. Bansal, J. Zhang, et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework,” 2023.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

- [13] A. Dubey et al., “The Llama 3 Herd of Models,” arXiv preprint arXiv:2407.21783, 2024.
- [14] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” in *Proceedings of EMNLP*, 2023.
- [15] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” arXiv preprint arXiv:2104.09864, 2021.
- [16] N. Shazeer, “GLU Variants Improve Transformer,” arXiv preprint arXiv:2002.05202, 2020.
- [17] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Advances in Neural Information Processing Systems*, 2019.